

Chapter 11. Securing Kafka

Kafka is used for a variety of use cases ranging from website activity tracking and metrics pipelines to patient record management and online payments. Each use case has different requirements in terms of security, performance, reliability, and availability. While it is always preferable to use the strongest and latest security features available, trade-offs are often necessary since increased security impacts performance, cost, and user experience. Kafka supports several standard security technologies with a range of configuration options to tailor security to each use case.

Like performance and reliability, security is an aspect of the system that must be addressed for the system as a whole, rather than component by component. The security of a system is only as strong as the weakest link, and security processes and policies must be enforced across the system, including the underlying platform. The customizable security features in Kafka enable integration with existing security infrastructure to build a consistent security model that applies to the entire system.

In this chapter, we will discuss the security features in Kafka and see how they address different aspects of security and contribute toward the overall security of the Kafka installation. Throughout the chapter, we will share best practices, potential threats, and techniques to mitigate these threats. We will also review additional measures that can be adopted to secure ZooKeeper and the rest of the platform.

Locking Down Kafka

Kafka uses a range of security procedures to establish and maintain confidentiality, integrity, and availability of data:

- Authentication establishes your identity and determines *who* you are.
- Authorization determines *what* you are allowed to do.
- Encryption protects your data from eavesdropping and tampering.
- Auditing tracks what you have done or have attempted to do.
- Quotas control how much resources you can utilize.

To understand how to lock down a Kafka deployment, let's first look at how data flows through a Kafka cluster. [Figure 11-1](#) shows the main steps in an example data flow. In this chapter, we will use this example flow to examine the different ways in which Kafka can be configured to protect data at every step to guarantee security of the entire deployment.

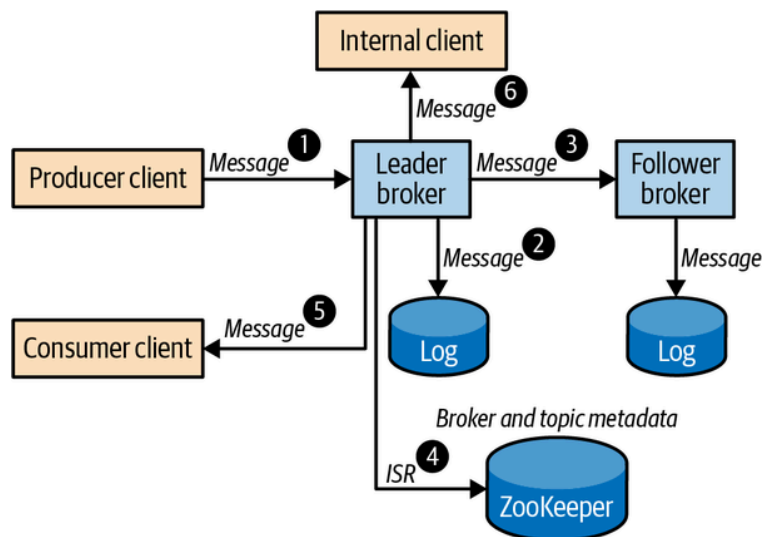


Figure 11-1. Data flow in a Kafka cluster

1. Alice produces a customer order record to a partition of the topic named `customerOrders`. The record is sent to the leader of the partition.
2. The leader broker writes the record to its local log file.
3. A follower broker fetches the message from the leader and writes to its local replica log file.
4. The leader broker updates the partition state in ZooKeeper to update in-sync replicas, if required.
5. Bob consumes customer order records from the topic `customerOrders`. Bob receives the record produced by Alice.
6. An internal application processes all messages arriving in `customerOrders` to produce real-time metrics on popular products.

A secure deployment must guarantee:

Client authenticity

When Alice establishes a client connection to the broker, the broker should authenticate the client to ensure that the message is really coming from Alice.

Server authenticity

Before sending a message to the leader broker, Alice's client should verify that the connection is to the real broker.

Data privacy

All connections where the message flows, as well as all disks where messages are stored, should be encrypted or physically secured to prevent eavesdroppers from reading the data and to ensure that data cannot be stolen.

Data integrity

Message digests should be included for data transmitted over insecure networks to detect tampering.

Access control

Before writing the message to the log, the leader broker should verify that Alice is authorized to write to `customerOrders`. Before returning messages to Bob's consumer, the broker should verify that Bob is authorized to read from the topic. If Bob's consumer uses group management, the broker should also verify that Bob has access to the consumer group.

Auditability

An audit trail that shows all operations that were performed by brokers, Alice, Bob, and other clients should be logged.

Availability

Brokers should apply quotas and limits to avoid some users hogging all the available bandwidth or overwhelming the broker with denial-of-service attacks. ZooKeeper should be locked down to ensure availability of the Kafka cluster since broker availability is dependent on ZooKeeper availability and the integrity of metadata stored in ZooKeeper.

In the following sections, we explore the Kafka security features that can be used to provide these guarantees. We first introduce the Kafka connection model and the security protocols associated with connections from clients to Kafka brokers. We then look at each security protocol in detail and examine the authentication capabilities of each protocol to ascertain client authenticity and server authenticity. We review options for encryption at different stages, including built-in encryption of data in transit in some security protocols to address data privacy and data integrity. Then, we explore customizable authorization in Kafka to manage access control and the main logs that contribute to auditability. Finally, we review security for the rest of the system, including ZooKeeper and the platform, which is necessary to maintain availability. For details on quotas that contribute to service availability through fair allocation of resources among users, refer to [Chapter 3](#).

Security Protocols

Kafka brokers are configured with listeners on one or more endpoints and accept client connections on these listeners. Each listener can be configured with its own security settings. Security requirements on a private internal listener that is physically protected and only accessible to authorized personnel may be different from the security requirements of an external listener accessible over the public internet. The choice of security protocol determines the level of authentication and encryption of data in transit.

Kafka supports four security protocols using two standard technologies, TLS and SASL. Transport Layer Security (TLS), commonly referred to by the name of its predecessor, Secure Sockets Layer (SSL), supports encryption as well as client and server authentication. Simple Authentication and Security Layer (SASL) is a framework for providing authentication using different mechanisms in connection-oriented protocols. Each Kafka security protocol combines a transport layer (PLAINTEXT or SSL) with an optional authentication layer (SSL or SASL):

PLAINTEXT

PLAINTEXT transport layer with no authentication. Is suitable only for use within private networks for processing data that is not sensitive since no authentication or encryption is used.

SSL

SSL transport layer with optional SSL client authentication. Is suitable for use in insecure networks since client and server authentication as well as encryption are supported.

SASL_PLAINTEXT

PLAINTEXT transport layer with SASL client authentication. Some SASL mechanisms also support server authentication. Does not support encryption and hence is suitable only for use within private networks.

`SASL_SSL`

SSL transport layer with SASL authentication. Is suitable for use in insecure networks since client and server authentication as well as encryption are supported.

`TLS/SSL`

TLS is one of the most widely used cryptographic protocols on the public internet. Application protocols like HTTP, SMTP, and FTP rely on TLS to provide privacy and integrity of data in transit. TLS relies on a Public Key Infrastructure (PKI) to create, manage, and distribute digital certificates that can be used for asymmetric encryption, avoiding the need for distributing shared secrets between servers and clients. Session keys generated during the TLS handshake enable symmetric encryption with higher performance for subsequent data transfer.

The listener used for inter-broker communication can be selected by configuring `inter.broker.listener.name` or `security.inter.broker.protocol`. Both server-side and client-side configuration options must be provided in the broker configuration for the security protocol used for inter-broker communication. This is because brokers need to establish client connections for that listener. The following example configures SSL for the inter-broker and internal listeners, and `SASL_SSL` for the external listener:

```
listeners=EXTERNAL://:9092,INTERNAL://10.0.0.2:9093,BROKER://10.0.0.2:9094
advertised.listeners=EXTERNAL://broker1.example.com:9092,INTERNAL://broker1.local:9093,B
listener.security.protocol.map=EXTERNAL:SASL_SSL,INTERNAL:SSL,BROKER:SSL
inter.broker.listener.name=BROKER
```

Clients are configured with a security protocol and bootstrap servers that determine the broker listener. Metadata returned to clients contains only the endpoints corresponding to the same listener as the bootstrap servers:

```
security.protocol=SASL_SSL
bootstrap.servers=broker1.example.com:9092,broker2.example.com:9092
```

In the next section on authentication, we review the protocol-specific configuration options for brokers and clients for each security protocol.

Authentication

Authentication is the process of establishing the identity of the client and server to verify client authenticity and server authenticity. When Alice's client connects to the leader broker to produce a customer order record, server authentication enables the client to establish that the server that the client is talking to is the actual broker. Client authentication verifies Alice's identity by validating Alice's credentials, like a password or digital certificate, to determine that the connection is from Alice and not an impersonator. Once authenticated, Alice's identity is associated with the connection throughout the lifetime of the connection. Kafka uses an instance of `KafkaPrincipal` to represent client identity and uses this

principal to grant access to resources and allocate quotas for connections with that client identity. The `KafkaPrincipal` for each connection is established during authentication based on the authentication protocol. For example, the principal `User:Alice` may be used for Alice based on the username provided for password-based authentication. `KafkaPrincipal` may be customized by configuring `principal.builder.class` for brokers.

ANONYMOUS CONNECTIONS

The principal `User:ANONYMOUS` is used for unauthenticated connections. This includes clients on PLAINTEXT listeners as well as unauthenticated clients on SSL listeners.

SSL

When Kafka is configured with SSL or SASL_SSL as the security protocol for a listener, TLS is used as the secure transport layer for connections on that listener. When a connection is established over TLS, the TLS handshake process performs authentication, negotiates cryptographic parameters, and generates shared keys for encryption. The server's digital certificate is verified by the client to establish the identity of the server. If client authentication using SSL is enabled, the server also verifies the client's digital certificate to establish the identity of the client. All traffic over SSL is encrypted, making it suitable for use in insecure networks.

SSL PERFORMANCE

SSL channels are encrypted and hence introduce a noticeable overhead in terms of CPU usage. Zero-copy transfer is currently not supported for SSL. Depending on the traffic pattern, the overhead may be up to 20–30%.

Configuring TLS

When TLS is enabled for a broker listener using SSL or SASL_SSL, brokers should be configured with a key store containing the broker's private key and certificate, and clients should be configured with a trust store containing the broker certificate or the certificate of the certificate authority (CA) that signed the broker certificate. Broker certificates should contain the broker hostname as a Subject Alternative Name (SAN) extension or as the Common Name (CN) to enable clients to verify the server hostname. Wildcard certificates can be used to simplify administration by using the same key store for all brokers in a domain.

SERVER HOSTNAME VERIFICATION

By default, Kafka clients verify that the hostname of the server stored in the server certificate matches the host that the client is connecting to. The connection hostname may be a bootstrap server that the client is configured with or an advertised listener hostname that was returned by a broker in a metadata response. Hostname verification is a critical part of server authentication that protects against man-in-the-middle attacks and hence should not be disabled in production systems.

Brokers can be configured to authenticate clients connecting over listeners using SSL as the security protocol by setting the broker configuration option `ssl.client.auth=required`. Clients should be configured with a key store, and brokers should be configured with a trust store contain-

ing client certificates or the certificate of the CAs that signed the client certificates. If SSL is used for inter-broker communication, broker trust stores should include the CA of the broker certificates as well as the CA of the client certificates. By default, the distinguished name (DN) of the client certificate is used as the `KafkaPrincipal` for authorization and quotas. The configuration option `ssl.principal.mapping.rules` can be used to provide a list of rules to customize the principal. Listeners using SASL_SSL disable TLS client authentication and rely on SASL authentication and the `KafkaPrincipal` established by SASL.

SSL CLIENT AUTHENTICATION

SSL client authentication may be made optional by setting `ssl.`

`client.auth=requested`. Clients that are not configured with key stores will complete the TLS handshake in this case, but will be assigned the principal `User:ANONYMOUS`.

The following examples show how to create key stores and trust stores for server and client authentication using a self-signed CA.

Generate self-signed CA key-pair for brokers:

```
$ keytool -genkeypair -keyalg RSA -keysize 2048 -keystore server.ca.p12 \
  -storetype PKCS12 -storepass server-ca-password -keypass server-ca-password \
  -alias ca -dname "CN=BrokerCA" -ext bc=ca:true -validity 365 ❶
$ keytool -export -file server.ca.crt -keystore server.ca.p12 \
  -storetype PKCS12 -storepass server-ca-password -alias ca -rfc ❷
```

- ❶ Create a key-pair for the CA and store it in a PKCS12 file `server-ca.p12`. We use this for signing certificates.
- ❷ Export the CA's public certificate to `server.ca.crt`. This will be included in trust stores and certificate chains.

Create key stores for brokers with a certificate signed by the self-signed CA. If using wildcard hostnames, the same key store can be used for all brokers. Otherwise, create a key store for each broker with its fully qualified domain name (FQDN):

```
$ keytool -genkey -keyalg RSA -keysize 2048 -keystore server.ks.p12 \
  -storepass server-ks-password -keypass server-ks-password -alias server \
  -storetype PKCS12 -dname "CN=Kafka,O=Confluent,C=GB" -validity 365 ❶
$ keytool -certreq -file server.csr -keystore server.ks.p12 -storetype PKCS12 \
  -storepass server-ks-password -keypass server-ks-password -alias server ❷
$ keytool -gencert -infile server.csr -outfile server.crt \
  -keystore server.ca.p12 -storetype PKCS12 -storepass server-ca-password \
  -alias ca -ext SAN=DNS:broker1.example.com -validity 365 ❸
$ cat server.crt server.ca.crt > serverchain.crt
$ keytool -importcert -file serverchain.crt -keystore server.ks.p12 \
  -storepass server-ks-password -keypass server-ks-password -alias server \
  -storetype PKCS12 -noprompt ❹
```

- ❶ Generate a private key for a broker and store it in the PKCS12 file `server.ks.p12`.
- ❷ Generate a certificate signing request.
- ❸ Use the CA key store to sign the broker's certificate. The signed certificate is stored in `server.crt`.

- 4 Import the broker's certificate chain into the broker's key store.

If TLS is used for inter-broker communication, create a trust store for brokers with the broker's CA certificate to enable brokers to authenticate one another:

```
$ keytool -import -file server.ca.crt -keystore server.ts.p12 \
  -storetype PKCS12 -storepass server-ts-password -alias server -noprompt
```

Generate a trust store for clients with the broker's CA certificate:

```
$ keytool -import -file server.ca.crt -keystore client.ts.p12 \
  -storetype PKCS12 -storepass client-ts-password -alias ca -noprompt
```

If TLS client authentication is enabled, clients must be configured with a key store. The following script generates a self-signed CA for clients and creates a key store for clients with a certificate signed by the client CA. The client CA is added to the broker trust store so that brokers can verify client authenticity:

```
# Generate self-signed CA key-pair for clients
keytool -genkeypair -keyalg RSA -keysize 2048 -keystore client.ca.p12 \
  -storetype PKCS12 -storepass client-ca-password -keypass client-ca-password \
  -alias ca -dname CN=ClientCA -ext bc=ca:true -validity 365 ❶
keytool -export -file client.ca.crt -keystore client.ca.p12 -storetype PKCS12 \
  -storepass client-ca-password -alias ca -rfc

# Create key store for clients
keytool -genkey -keyalg RSA -keysize 2048 -keystore client.ks.p12 \
  -storepass client-ks-password -keypass client-ks-password -alias client \
  -storetype PKCS12 -dname "CN=Metrics App,O=Confluent,C=GB" -validity 365 ❷
keytool -certreq -file client.csr -keystore client.ks.p12 -storetype PKCS12 \
  -storepass client-ks-password -keypass client-ks-password -alias client
keytool -gencert -infile client.csr -outfile client.crt \
  -keystore client.ca.p12 -storetype PKCS12 -storepass client-ca-password \
  -alias ca -validity 365
cat client.crt client.ca.crt > clientchain.crt
keytool -importcert -file clientchain.crt -keystore client.ks.p12 \
  -storepass client-ks-password -keypass client-ks-password -alias client \
  -storetype PKCS12 -noprompt ❸

# Add client CA certificate to broker's trust store
keytool -import -file client.ca.crt -keystore server.ts.p12 -alias client \
  -storetype PKCS12 -storepass server-ts-password -noprompt ❹
```

- ❶ We create a new CA for clients in this example.
- ❷ Clients authenticating with this certificate use `User:CN=Metrics App,O=Confluent,C=GB` as the principal, by default.
- ❸ We add the client certificate chain to the client key store.
- ❹ The broker's trust store should contain the CAs of all clients.

Once we have the key and trust stores, we can configure TLS for brokers. Brokers require a trust store only if TLS is used for inter-broker communication or if client authentication is enabled:

```
ssl.keystore.location=/path/to/server.ks.p12
ssl.keystore.password=server-ks-password
ssl.key.password=server-ks-password
```

```
ssl.keystore.type=PKCS12
ssl.truststore.location=/path/to/server.ts.p12
ssl.truststore.password=server-ts-password
ssl.truststore.type=PKCS12
ssl.client.auth=required
```

Clients are configured with the generated trust store. The key store should be configured for clients if client authentication is required.

```
ssl.truststore.location=/path/to/client.ts.p12
ssl.truststore.password=client-ts-password
ssl.truststore.type=PKCS12
ssl.keystore.location=/path/to/client.ks.p12
ssl.keystore.password=client-ks-password
ssl.key.password=client-ks-password
ssl.keystore.type=PKCS12
```

TRUST STORES

Trust store configuration can be omitted in brokers as well as clients when using certificates signed by well-known trusted authorities. The default trust stores in the Java installation will be sufficient to establish trust in this case. Installation steps are described in [Chapter 2](#).

Key stores and trust stores must be updated periodically before certificates expire to avoid TLS handshake failures. Broker SSL stores can be dynamically updated by modifying the same file or setting the configuration option to a new versioned file. In both cases, the Admin API or the Kafka configs tool can be used to trigger the update. The following example updates the key store for the external listener of a broker with broker id 0 using the configs tool:

```
$ bin/kafka-configs.sh --bootstrap-server localhost:9092 \
  --command-config admin.props \
  --entity-type brokers --entity-name 0 --alter --add-config \
  'listener.name.external.ssl.keystore.location=/path/to/server.ks.p12'
```

Security considerations

TLS is widely used to provide transport layer security for several protocols, including HTTPS. As with any security protocol, it is important to understand the potential threats and mitigation strategies when adopting a protocol for mission-critical applications. Kafka enables only the newer protocols TLSv1.2 and TLSv1.3 by default, since older protocols like TLSv1.1 have known vulnerabilities. Due to issues with insecure renegotiation, Kafka does not support renegotiation for TLS connections. Hostname verification is enabled by default to prevent man-in-the-middle attacks. Security can be tightened further by restricting cipher suites. Strong ciphers with at least a 256-bit encryption key size protect against cryptographic attacks and ensure data integrity when transporting data over an insecure network. Some organizations require TLS protocol and ciphers to be restricted to comply with security standards like FIPS 140-2.

Since key stores containing private keys are stored on the filesystem by default, it is vital to limit access to key store files using filesystem permissions. Standard Java TLS features can be used to enable certificate revocation if a private key is compromised. Short-lived keys can be used to reduce exposure in this case.

TLS handshakes are expensive and utilize a significant amount of time on network threads in brokers. Listeners using TLS on insecure networks should be protected against denial-of-service attacks using connection quotas and limits to protect availability of brokers. The broker configuration option `connection.failed.authentication.delay.ms` can be used to delay failed response on authentication failures to reduce the rate at which authentication failures are retried by clients.

SASL

Kafka protocol supports authentication using SASL and has built-in support for several commonly used SASL mechanisms. SASL can be combined with TLS as the transport layer to provide a secure channel with authentication and encryption. SASL authentication is performed through a sequence of server challenges and client responses where the SASL mechanism defines the sequence and wire format of challenges and responses. Kafka brokers support the following SASL mechanisms out of the box with customizable callbacks to integrate with existing security infrastructure:

GSSAPI

Kerberos authentication is supported using SASL/GSSAPI and can be used to integrate with Kerberos servers like Active Directory or OpenLDAP.

PLAIN

Username/password authentication that is typically used with a custom server-side callback to verify passwords from an external password store.

SCRAM-SHA-256 and SCRAM-SHA-512

Username/password authentication available out of the box with Kafka without the need for additional password stores.

OAUTHBEARER

Authentication using OAuth bearer tokens that is typically used with custom callbacks to acquire and validate tokens granted by standard OAuth servers.

One or more SASL mechanisms may be enabled on each SASL-enabled listener in the broker by configuring `sasl.enabled.mechanisms` for that listener. Clients may choose any of the enabled mechanisms by configuring `sasl.mechanism`.

Kafka uses the Java Authentication and Authorization Service (JAAS) for configuring SASL. The configuration option `sasl.jaas.config` contains a single JAAS configuration entry that specifies a login module and its options. Brokers use the `listener` and `mechanism` prefixes when configuring `sasl.jaas.config`. For example, `listener.name.external.gssapi.sasl.jaas.config` configures the JAAS configuration entry for SASL/GSSAPI on the listener named `EXTERNAL`. The login process on brokers and clients uses the JAAS configuration to determine the public and private credentials used for authentication.

JAAS configuration may also be specified in configuration files using the Java system property `java.security.auth.login.config`. However, the Kafka option `sasl.jaas.config` is recommended since it supports password protection and separate configuration for each SASL mechanism when multiple mechanisms are enabled on a listener.

SASL mechanisms supported by Kafka can be customized to integrate with third-party authentication servers using callback handlers. A login callback handler may be provided for brokers or clients to customize the login process, for example, to acquire credentials to be used for authentication. A server callback handler may be provided to perform authentication of client credentials, for example, to verify passwords using an external password server. A client callback handler may be provided to inject client credentials instead of including them in the JAAS configuration.

In the following subsections, we explore the SASL mechanisms supported by Kafka in more detail.

SASL/GSSAPI

Kerberos is a widely used network authentication protocol that uses strong cryptography to support secure mutual authentication over an insecure network. Generic Security Service Application Program Interface (GSS-API) is a framework for providing security services to applications using different authentication mechanisms. [RFC-4752](#) introduces the SASL mechanism GSSAPI for authentication using GSS-API's Kerberos V5 mechanism. The availability of open source as well as enterprise-grade commercial implementations of Kerberos servers has made Kerberos a popular choice for authentication across many sectors with strict security requirements. Kafka supports Kerberos authentication using SASL/GSSAPI.

Configuring SASL/GSSAPI

Kafka uses GSSAPI security providers included in the Java runtime environment to support secure authentication using Kerberos. JAAS configuration for GSSAPI includes the path of a keytab file that contains the mapping of principals to their long-term keys in encrypted form. To configure GSSAPI for brokers, create a keytab for each broker with a principal that includes the broker's hostname. Broker hostnames are verified by clients to ensure server authenticity and prevent man-in-the-middle attacks. Kerberos requires a secure DNS service for host name lookup during authentication. In deployments where forward and reverse lookup do not match, the Kerberos configuration file `krb5.conf` on clients can be configured to set `rdns=false` to disable reverse lookup. JAAS configuration for each broker should include the Kerberos V5 login module from the Java runtime, the pathname of the keytab file, and the full broker principal:

```
sasl.enabled.mechanisms=GSSAPI
listener.name.external.gssapi.sasl.jaas.config=\ ❶
com.sun.security.auth.module.Krb5LoginModule required \
  useKeyTab=true storeKey=true \
  keyTab="/path/to/broker1.keytab" \ ❷
principal="kafka/broker1.example.com@EXAMPLE.COM"; ❸
```

- ❶ We use `sasl.jaas.config` prefixed with the listener prefix, which contains the listener name and SASL mechanism in

lowercase.

- ❷ Keytab files must be readable by the broker process.
- ❸ Service principal for brokers should include the broker hostname.

If SASL/GSSAPI is used for inter-broker communication, inter-broker SASL mechanism and the Kerberos service name should also be configured for brokers:

```
sasl.mechanism.inter.broker.protocol=GSSAPI
sasl.kerberos.service.name=kafka
```

Clients should be configured with their own keytab and principal in the JAAS configuration and `sasl.kerberos.service.name` to indicate the name of the service they are connecting to:

```
sasl.mechanism=GSSAPI
sasl.kerberos.service.name=kafka ❶
sasl.jaas.config=com.sun.security.auth.module.Krb5LoginModule required \
    useKeyTab=true storeKey=true \
    keyTab="/path/to/alice.keytab" \
    principal="Alice@EXAMPLE.COM"; ❷
```

- ❶ The service name for the Kafka service should be specified for clients.
- ❷ Clients may use principals without hostname.

The short name of the principal is used as the client identity by default. For example, `User:Alice` is the client principal and `User:kafka` is the broker principal in the example. The broker configuration `sasl.kerberos.principal.to.local.rules` can be used to apply a list of rules to transform the fully qualified principal to a custom principal.

Security considerations

Use of SASL_SSL is recommended in production deployments using Kerberos to protect the authentication flow as well as data traffic on the connection after authentication. If TLS is not used to provide a secure transport layer, eavesdroppers on the network may gain enough information to mount a dictionary attack or brute-force attack to steal client credentials. It is safer to use randomly generated keys for brokers instead of keys generated from passwords that are easier to crack. Weak encryption algorithms like DES-MD5 should be avoided in favor of stronger algorithms. Access to keytab files must be restricted using filesystem permissions since any user in possession of the file may impersonate the user.

SASL/GSSAPI requires a secure DNS service for server authentication. Because denial-of-service attacks against the KDC or DNS service can result in authentication failures in clients, it is necessary to monitor the availability of these services. Kerberos also relies on loosely synchronized clocks with configurable variability to detect replay attacks. It is important to ensure that clock synchronization is secure.

SASL/PLAIN

[RFC-4616](#) defines a simple username/password authentication mechanism that can be used with TLS to provide secure authentication. During

authentication, the client sends a username and password to the server, and the server verifies the password using its password store. Kafka has built-in SASL/PLAIN support that can be integrated with a secure external password database using a custom callback handler.

Configuring SASL/PLAIN

The default implementation of SASL/PLAIN uses the broker's JAAS configuration as the password store. All client usernames and passwords are included as login options, and the broker verifies that the password provided by a client during authentication matches one of these entries. A broker username and password are required only if SASL/PLAIN is used for inter-broker communication:

```
sasl.enabled.mechanisms=PLAIN
sasl.mechanism.inter.broker.protocol=PLAIN
listener.name.external.plain.sasl.jaas.config=\
  org.apache.kafka.common.security.plain.PlainLoginModule required \
    username="kafka" password="kafka-password" \ ❶
    user_kafka="kafka-password" \
    user_Alice="Alice-password"; ❷
```

- ❶ The username and password used for inter-broker connections initiated by the broker.
- ❷ When Alice's client connects to the broker, the password provided by Alice is validated against this password in the broker's config.

Clients must be configured with username and password for authentication:

```
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule \
  required username="Alice" password="Alice-password";
```

The built-in implementation that stores all passwords in every broker's JAAS configuration is insecure and not very flexible since all brokers will need to be restarted to add or remove a user. When using SASL/PLAIN in production, a custom server callback handler can be used to integrate brokers with a secure third-party password server. Custom callback handlers can also be used to support password rotation. On the server side, a server callback handler should support both old and new passwords for an overlapping period until all clients switch to the new password. The following example shows a callback handler that verifies encrypted passwords from files generated using the Apache tool `htpasswd`:

```
public class PasswordVerifier extends PlainServerCallbackHandler {

    private final List<String> passwdFiles = new ArrayList<>(); ❶

    @Override
    public void configure(Map<String, ?> configs, String mechanism,
                        List<AppConfigurationEntry> jaasEntries) {
        Map<String, ?> loginOptions = jaasEntries.get(0).getOptions();
        String files = (String) loginOptions.get("password.files"); ❷
        Collections.addAll(passwdFiles, files.split(", "));
    }

    @Override
    protected boolean authenticate(String user, char[] password) {
        return passwdFiles.stream() ❸
```

```

        .anyMatch(file -> authenticate(file, user, password));
    }

    private boolean authenticate(String file, String user, char[] password) {
        try {
            String cmd = String.format("htpasswd -vb %s %s %s",
                                     ❷      ❸      ❹
                                     file, user, new String(password));
            return Runtime.getRuntime().exec(cmd).waitFor() == 0;
        } catch (Exception e) {
            return false;
        }
    }
}

```

- ❶ We use multiple password files so that we can support password rotation.
- ❷ We pass pathnames of password files as a JAAS option in the broker configuration. Custom broker configuration options may also be used.
- ❸ We check if the password matches in any of the files, allowing both old and new passwords to be used for a period of time.
- ❹ We use `htpasswd` for simplicity. A secure database can be used for production deployments.

Brokers are configured with the password validation callback handler and its options:

```

listener.name.external.plain.sasl.jaas.config=\
    org.apache.kafka.common.security.plain.PlainLoginModule required \
    password.files="/path/to/htpassword.props,/path/to/oldhtpassword.props";
listener.name.external.plain.sasl.server.callback.handler.class=\
    com.example.PasswordVerifier

```

On the client side, a client callback handler that implements

`org.apache.kafka.`

`common.security.auth.AuthenticateCallbackHandler` can be used to load passwords dynamically at runtime when a connection is established instead of loading statically from the JAAS configuration during startup. Passwords may be loaded from encrypted files or using an external secure server to improve security. The following example loads passwords dynamically from a file using configuration classes in Kafka:

```

@Override
public void handle(Callback[] callbacks) throws IOException {
    Properties props = Uutils.loadProps(passwdFile);
    PasswordConfig config = new PasswordConfig(props);
    String user = config.getString("username");
    String password = config.getPassword("password").value();
    for (Callback callback: callbacks) {
        if (callback instanceof NameCallback)
            ((NameCallback) callback).setName(user);
        else if (callback instanceof PasswordCallback) {
            ((PasswordCallback) callback).setPassword(password.toCharArray());
        }
    }
}

private static class PasswordConfig extends AbstractConfig {
    static ConfigDef CONFIG = new ConfigDef()
        .define("username", STRING, HIGH, "User name")

```

```

        .define("password", PASSWORD, HIGH, "User password"); ❸
        PasswordConfig(Properties props) {
            super(CONFIG, props, false);
        }
    }
}

```

- ❶ We load the config file within the callback to ensure we use the latest password to support password rotation.
- ❷ The underlying configuration library returns the actual password value even if the password is externalized.
- ❸ We define password configs with the `PASSWORD` type to ensure that passwords are not included in log entries.

Clients as well as brokers that use SASL/PLAIN for inter-broker communication can be configured with the client-side callback:

```

sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule \
    required file="/path/to/credentials.props";
sasl.client.callback.handler.class=com.example.PasswordProvider

```

Security considerations

Since SASL/PLAIN transmits clear-text passwords over the wire, the PLAIN mechanism should be enabled only with encryption using SASL_SSL to provide a secure transport layer. Passwords stored in clear text in the JAAS configuration of brokers and clients are not secure, so consider encrypting or externalizing these passwords in a secure password store. Instead of using the built-in password store that stores all client passwords in the broker JAAS configuration, use a secure external password server that stores passwords securely and enforces strong password policies.

CLEAR-TEXT PASSWORDS

Avoid clear-text passwords in configuration files even if the files can be protected using filesystem permissions. Consider externalizing or encrypting passwords to ensure that passwords are not inadvertently exposed. Kafka's password protection feature is described later in this chapter.

SASL/SCRAM

[RFC-5802](#) introduces a secure username/password authentication mechanism that addresses the security concerns with password authentication mechanisms like SASL/PLAIN, which send passwords over the wire. The Salted Challenge Response Authentication Mechanism (SCRAM) avoids transmitting clear-text passwords and stores passwords in a format that makes it impractical to impersonate clients. Salting combines passwords with some random data before applying a one-way cryptographic hash function to store passwords securely. Kafka has a built-in SCRAM provider that can be used in deployments with secure ZooKeeper without the need for additional password servers. The SCRAM mechanisms `SCRAM-SHA-256` and `SCRAM-SHA-512` are supported by the Kafka provider.

Configuring SASL/SCRAM

An initial set of users can be created after starting ZooKeeper prior to starting brokers. Brokers load SCRAM user metadata into an in-memory

cache during startup, ensuring that all users, including the broker user for inter-broker communication, can authenticate successfully. Users can be added or deleted at any time. Brokers keep the cache up-to-date using notifications based on a ZooKeeper watcher. In this example, we create a user with the principal `User:Alice` and password `Alice-password` for SASL mechanism `SCRAM-SHA-512`:

```
$ bin/kafka-configs.sh --zookeeper localhost:2181 --alter --add-config \
  'SCRAM-SHA-512=[iterations=8192,password=Alice-password]' \
  --entity-type users --entity-name Alice
```

One or more SCRAM mechanisms can be enabled on a listener by configuring the mechanisms on the broker. A username and password are required for brokers only if the listener is used for inter-broker communication:

```
sasl.enabled.mechanisms=SCRAM-SHA-512
sasl.mechanism.inter.broker.protocol=SCRAM-SHA-512
listener.name.external.scram-sha-512.sasl.jaas.config=\
  org.apache.kafka.common.security.scram.ScramLoginModule required \
    username="kafka" password="kafka-password"; ❶
```

- ❶ Username and password for inter-broker connections initiated by the broker.

Clients must be configured to use one of the SASL mechanisms enabled in the broker, and the client JAAS configuration must include a username and password:

```
sasl.mechanism=SCRAM-SHA-512
sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLoginModule \
  required username="Alice" password="Alice-password";
```

You can add new SCRAM users using `--add-config` and delete users using the `--delete-config` option of the configs tool. When an existing user is deleted, new connections cannot be established for that user, but existing connections of the user will continue to work. A reauthentication interval can be configured for the broker to limit the amount of time existing connections may continue to operate after a user is deleted. The following example deletes the `SCRAM-SHA-512` config for `Alice` to remove Alice's credentials for that mechanism:

```
$ bin/kafka-configs.sh --zookeeper localhost:2181 --alter --delete-config \
  'SCRAM-SHA-512' --entity-type users --entity-name Alice
```

Security considerations

SCRAM applies a one-way cryptographic hash function on the password combined with a random salt to avoid the actual password being transmitted over the wire or stored in a database. However, any password-based system is only as secure as the passwords. Strong password policies must be enforced to protect the system from brute-force or dictionary attacks. Kafka provides safeguards by supporting only the strong hashing algorithms SHA-256 and SHA-512 and avoiding weaker algorithms like SHA-1. This is combined with a high default iteration count of 4,096 and unique random salts for every stored key to limit the impact if ZooKeeper security is compromised.

You should take additional precautions to protect the keys transmitted during handshake and the keys stored in ZooKeeper to protect against brute-force attacks. SCRAM must be used with `SASL_SSL` as the security protocol to avoid eavesdroppers from gaining access to hashed keys during authentication. ZooKeeper must also be SSL-enabled, and ZooKeeper data must be protected using disk encryption to ensure that stored keys cannot be retrieved even if the store is compromised. In deployments without a secure ZooKeeper, SCRAM callbacks can be used to integrate with a secure external credential store.

SASL/OAUTHBEARER

OAuth is an authorization framework that enables applications to obtain limited access to HTTP services. [RFC-7628](#) defines the OAUTHBEARER SASL mechanism that enables credentials obtained using OAuth 2.0 to access protected resources in non-HTTP protocols. OAUTHBEARER avoids security vulnerabilities in mechanisms that use long-term passwords by using OAuth 2.0 bearer tokens with a shorter lifetime and limited resource access. Kafka supports SASL/OAUTHBEARER for client authentication, enabling integration with third-party OAuth servers. The built-in implementation of OAUTHBEARER uses unsecured JSON Web Tokens (JWTs) and is not suitable for production use. Custom callbacks can be added to integrate with standard OAuth servers to provide secure authentication using the OAUTHBEARER mechanism in production deployments.

Configuring SASL/OAUTHBEARER

The built-in implementation of SASL/OAUTHBEARER in Kafka does not validate tokens and hence only requires the login module to be specified in the JAAS configuration. If the listener is used for inter-broker communication, details of the token used for client connections initiated by brokers must also be provided. The option `unsecuredLoginStringClaim_sub` is the subject claim that determines the `KafkaPrincipal` for the connection by default:

```
sasl.enabled.mechanisms=OAUTHBEARER
sasl.mechanism.inter.broker.protocol=OAUTHBEARER
listener.name.external.oauthbearer.sasl.jaas.config=\
    org.apache.kafka.common.security.oauthbearer.OAuthBearerLoginModule \
        required unsecuredLoginStringClaim_sub="kafka"; ❶
```

- ❶ Subject claim for the token used for inter-broker connections.

Clients must be configured with the subject claim option `unsecuredLoginStringClaim_sub`. Other claims and token lifetime may also be configured:

```
sasl.mechanism=OAUTHBEARER
sasl.jaas.config=\
    org.apache.kafka.common.security.oauthbearer.OAuthBearerLoginModule \
        required unsecuredLoginStringClaim_sub="Alice"; ❶
```

- ❶ `User:Alice` is the default `KafkaPrincipal` for connections using this configuration.

To integrate Kafka with third-party OAuth servers for using bearer tokens in production, Kafka clients must be configured with `sasl.login.callback.handler.class` to acquire tokens from the OAuth server using the long-term password or a refresh token. If

OAuthBEARER is used for inter-broker communication, brokers must also be configured with a login callback handler to acquire tokens for client connections created by the broker for inter-broker communication:

```
@Override
public void handle(Callback[] callbacks) throws UnsupportedOperationException {
    OAuthBearerToken token = null;
    for (Callback callback : callbacks) {
        if (callback instanceof OAuthBearerTokenCallback) {
            token = acquireToken(); ❶
            ((OAuthBearerTokenCallback) callback).token(token);
        } else if (callback instanceof SaslExtensionsCallback) { ❷
            ((SaslExtensionsCallback) callback).extensions(processExtensions(token));
        } else
            throw new UnsupportedOperationException(callback);
    }
}
```

❶ Clients must acquire a token from the OAuth server and set a valid token on the callback.

❷ The client may also include optional extensions.

Brokers must also be configured with a server callback handler using `listener.name.<listener-name>.oauthbearer.sasl.server.callback.handler.class` for validating tokens provided by the client:

```
@Override
public void handle(Callback[] callbacks) throws UnsupportedOperationException {
    for (Callback callback : callbacks) {
        if (callback instanceof OAuthBearerValidatorCallback) {
            OAuthBearerValidatorCallback cb = (OAuthBearerValidatorCallback) callback;
            try {
                cb.token(validatedToken(cb.tokenValue())); ❶
            } catch (OAuthBearerIllegalTokenException e) {
                OAuthBearerValidationResult r = e.reason();
                cb.error(errorStatus(r), r.failureScope(), r.failureOpenIdConfig());
            }
        } else if (callback instanceof OAuthBearerExtensionsValidatorCallback) {
            OAuthBearerExtensionsValidatorCallback ecb =
                (OAuthBearerExtensionsValidatorCallback) callback;
            ecb.inputExtensions().map().forEach((k, v) ->
                ecb.valid(validateExtension(k, v))); ❷
        } else {
            throw new UnsupportedOperationException(callback);
        }
    }
}
```

❶ `OAuthBearerValidatorCallback` contains the token from the client. Brokers validate this token.

❷ Brokers validate any optional extensions from the client.

Security considerations

Since SASL/OAuthBEARER clients send OAuth 2.0 bearer tokens over the network and these tokens may be used to impersonate clients, TLS must be enabled to encrypt authentication traffic. Short-lived tokens can be used to limit exposure if tokens are compromised. Reauthentication may be enabled for brokers to prevent connections outliving the tokens used for authentication. A reauthentication interval configured on brokers,

combined with token revocation support, limit the amount of time an existing connection may continue to use a token after revocation.

Delegation tokens

Delegation tokens are shared secrets between Kafka brokers and clients that provide a lightweight configuration mechanism without the requirement to distribute SSL key stores or Kerberos keytabs to client applications. Delegation tokens can be used to reduce the load on authentication servers, like the Kerberos Key Distribution Center (KDC). Frameworks like Kafka Connect can use delegation tokens to simplify security configuration for workers. A client that has authenticated with Kafka brokers can create delegation tokens for the same user principal and distribute these tokens to workers, which can then authenticate directly with Kafka brokers. Each delegation token consists of a token identifier and a hash-based message authentication code (HMAC) used as a shared secret. Client authentication with delegation tokens is performed using SASL/SCRAM with the token identifier as username and HMAC as the password.

Delegation tokens can be created or renewed using the Kafka Admin API or the `delegation-tokens` command. To create delegation tokens for the principal `User:Alice`, the client must be authenticated using Alice's credentials for any authentication protocol other than delegation tokens. Clients authenticated using delegation tokens cannot create other delegation tokens:

```
$ bin/kafka-delegation-tokens.sh --bootstrap-server localhost:9092 \
  --command-config admin.props --create --max-life-time-period -1 \
  --renewer-principal User:Bob ❶
$ bin/kafka-delegation-tokens.sh --bootstrap-server localhost:9092 \
  --command-config admin.props --renew --renew-time-period -1 --hmac c2VjcmV0 ❷
```

- ❶ If Alice runs this command, the generated token can be used to impersonate Alice. The owner of this token is `User:Alice`. We also configure `User:Bob` as a token renewer.
- ❷ The renewal command can be run by the token owner (Alice) or the token renewer (Bob).

Configuring delegation tokens

To create and validate delegation tokens, all brokers must be configured with the same master key using the configuration option `delegation.token.master.key`. This key can only be rotated by restarting all brokers. All existing tokens should be deleted before updating the master key since they can no longer be used, and new tokens should be created after the key is updated on all brokers.

At least one of the SASL/SCRAM mechanisms must be enabled on brokers to support authentication using delegation tokens. Clients should be configured to use SCRAM with a token identifier as username and token HMAC as the password. The `kafkaPrincipal` for the connections using this configuration will be the original principal associated with the token, e.g., `User:Alice`:

```
sasl.mechanism=SCRAM-SHA-512
sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLoginModule \
  required tokenauth="true" username="MTIz" password="c2VjcmV0"; ❶
```

- ❶ SCRAM configuration with `tokenauth` is used to configure delegation tokens.

Security considerations

Like the built-in SCRAM implementation, delegation tokens are suitable for production use only in deployments where ZooKeeper is secure. All the security considerations described under SCRAM also apply to delegation tokens.

The master key used by brokers for generating tokens must be protected using encryption or by externalizing the key in a secure password store. Short-lived delegation tokens can be used to limit exposure if a token is compromised. Reauthentication can be enabled in brokers to prevent connections operating with expired tokens and to limit the amount of time existing connections may continue to operate after token deletion.

Reauthentication

As we saw earlier, Kafka brokers perform client authentication when a connection is established by the client. Client credentials are verified by the brokers, and the connection authenticates successfully if the credentials are valid at that time. Some security mechanisms like Kerberos and OAuth use credentials with a limited lifetime. Kafka uses a background login thread to acquire new credentials before the old ones expire, but the new credentials are used only to authenticate new connections by default. Existing connections that were authenticated with old credentials continue to process requests until disconnection occurs due to a request timeout, an idle timeout, or network errors. Long-lived connections may continue to process requests long after the credentials used to authenticate the connections expire. Kafka brokers support reauthentication for connections authenticated using SASL using the configuration option `connections.max.reauth.ms`. When this option is set to a positive integer, Kafka brokers determine the session lifetime for SASL connections and inform clients of this lifetime during the SASL handshake. Session lifetime is the lower of the remaining lifetime of the credential or `connections.max.reauth.ms`. Any connection that doesn't reauthenticate within this interval is terminated by the broker. Clients perform reauthentication using the latest credentials acquired by the background login thread or injected using custom callbacks. Reauthentication can be used to tighten security in several scenarios:

- For SASL mechanisms like GSSAPI and OAUTHBEARER that use credentials with a limited lifetime, reauthentication guarantees that all active connections are associated with valid credentials. Short-lived credentials limit exposure in case credentials that are compromised.
- Password-based SASL mechanisms like PLAIN and SCRAM can support password rotation by adding periodic login. Reauthentication limits the amount of time requests are processed on connections authenticated with the old password. Custom server callback that allows both old and new passwords for a period of time can be used to avoid outages until all clients migrate to the new password.
- `connections.max.reauth.ms` forces reauthentication in all SASL mechanisms, including those with nonexpiring credentials. This limits the amount of time a credential may be associated with an active connection after it has been revoked.
- Connections from clients without SASL reauthentication support are terminated on session expiry, forcing the clients to reconnect and au-

thenticate again, thus providing the same security guarantees for expired or revoked credentials.

COMPROMISED USERS

If a user is compromised, action must be taken to remove the user from the system as soon as possible. All new connections will fail to authenticate with Kafka brokers once the user is removed from the authentication server. Existing connections will continue to process requests until the next reauthentication timeout. If `connections.max.reauth.ms` is not configured, no timeout is applied and existing connections may continue to use the compromised user's identity for a long time. Kafka does not support SSL renegotiation due to known vulnerabilities during renegotiation in older SSL protocols. Newer protocols like TLSv1.3 do not support renegotiation. So, existing SSL connections may continue to use revoked or expired certificates. `Deny` ACLs for the user principal can be used to prevent these connections from performing any operation. Since ACL changes are applied with very small latencies across all brokers, this is the quickest way to disable access for compromised users.

Security Updates Without Downtime

Kafka deployments need regular maintenance to rotate secrets, apply security fixes, and update to the latest security protocols. Many of these maintenance tasks are performed using rolling updates where one by one, brokers are shut down and restarted with an updated configuration. Some tasks like updating SSL key stores and trust stores can be performed using dynamic config updates without restarting brokers.

When adding a new security protocol to an existing deployment, a new listener can be added to brokers with the new protocol while retaining the old listener with the old protocol to ensure that client applications can continue to function using the old listener during the update. For example, the following sequence can be used to switch from PLAINTEXT to SASL_SSL in an existing deployment:

1. Add a new listener on a new port to each broker using the Kafka configs tool. Use a single config update command to update `listeners` and `advertised.listeners` to include the old listener as well as the new listener, and provide all the configuration options for the new SASL_SSL listener with the listener prefix.
2. Modify all client applications to use the new SASL_SSL listener.
3. If inter-broker communication is being updated to use the new SASL_SSL listener, perform a rolling update of brokers with the new `inter.broker.listener.name`.
4. Use the configs tool to remove the old listener from `listeners` and `advertised.listeners` and to remove any unused configuration options of the old listener.

SASL mechanisms can be added or removed from existing SASL listeners without downtime using rolling updates on the same listener port. The following sequence switches the mechanism from PLAIN to SCRAM-SHA-256:

1. Add all existing users to the SCRAM store using the Kafka configs tool.
2. Set `sasl.enabled.mechanisms=PLAIN,SCRAM-SHA-256`, configure `listener.name.<_listener-name>.scram-sha-256.sasl.jaas.config` for the listener, and perform a rolling update of brokers.
3. Modify all client applications to use `sasl.mechanism=SCRAM-SHA-256` and update `sasl.jaas.config` to use SCRAM.

4. If the listener is used for inter-broker communication, use a rolling update of brokers to set
`sasl.mechanism.inter.broker.protocol=SCRAM-SHA-256`.
5. Perform another rolling update of brokers to remove the PLAIN mechanism. Set `sasl.enabled.mechanisms=SCRAM-SHA-256` and remove `listener.name.<listener-name>.plain.sasl.jaas.config` and any other configuration options for PLAIN.

Encryption

Encryption is used to preserve data privacy and data integrity. As we discussed earlier, Kafka listeners using SSL and SASL_SSL security protocols use TLS as the transport layer, providing secure encrypted channels that protect data transmitted over an insecure network. TLS cipher suites can be restricted to strengthen security and adhere to security requirements like the Federal Information Processing Standard (FIPS).

Additional measures must be taken to protect data at rest to ensure that sensitive data cannot be retrieved even by users with physical access to the disk that stores Kafka logs. To avoid security breaches even if the disk is stolen, physical storage can be encrypted using whole disk encryption or volume encryption.

While encryption of transport layer and data storage may provide adequate protection in many deployments, additional protection may be required to avoid granting automatic data access to platform administrators. Unencrypted data present in broker memory may appear in heap dumps, and administrators with direct access to the disk will be able to access these, as well as Kafka logs containing potentially sensitive data. In deployments with highly sensitive data or Personally Identifiable Information (PII), extra measures are required to preserve data privacy. To comply with regulatory requirements, especially in cloud deployments, it is necessary to guarantee that confidential data cannot be accessed by platform administrators or cloud providers by any means. Custom encryption providers can be plugged into Kafka clients to implement end-to-end encryption that guarantees that the entire data flow is encrypted.

End-to-End Encryption

In [Chapter 3](#) on Kafka producers, we saw that *serializers* are used to convert messages into the byte array stored in Kafka logs, and in [Chapter 4](#) on Kafka consumers, we saw that *deserializers* converted the byte array back to the message. Serializers and deserializers can be integrated with an encryption library to perform encryption of the message during serialization, and decryption during deserialization. Message encryption is typically performed using symmetric encryption algorithms like AES. A shared encryption key stored in a key management system (KMS) enables producers to encrypt the message and consumers to decrypt the message. Brokers do not require access to the encryption key and never see the unencrypted contents of the message, making this approach safe to use in cloud environments. Encryption parameters that are required to decrypt the message may be stored in message headers or in the message payload if older consumers without header support need access to the message. A digital signature may also be included in message headers to verify message integrity.

[Figure 11-2](#) shows a Kafka data flow with end-to-end encryption.

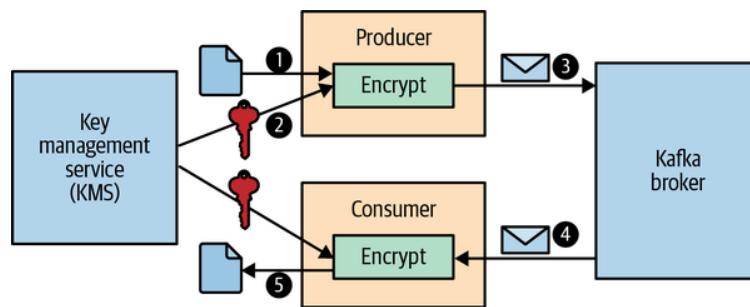


Figure 11-2. End-to-end encryption

1. We send a message using a Kafka producer.
2. The producer uses an encryption key from KMS to encrypt the message.
3. The encrypted message is sent to the broker. The broker stores the encrypted message in the partition logs.
4. The broker sends the encrypted message to consumers.
5. The consumer uses the encryption key from KMS to decrypt the message.

Producers and consumers must be configured with credentials to obtain shared keys from KMS. Periodic key rotation is recommended to harden security, since frequent rotation limits the number of compromised messages in case of a breach and also protects against brute-force attacks. Consumption must be supported with both old and new keys during the retention period of messages encrypted with the old key. Many KMS systems support graceful key rotation out of the box for symmetric encryption without requiring any special handling in Kafka clients. For compacted topics, messages encrypted with old keys may be retained for a long time, and it may be necessary to re-encrypt old messages. To avoid interference with newer messages, producers and consumers must be offline during this process.

COMPRESSION OF ENCRYPTED MESSAGES

Compressing messages after encryption is unlikely to provide any benefit in terms of space reduction compared to compressing prior to encryption. Serializers may be configured to perform compression before encrypting the message, or applications may be configured to perform compression prior to producing messages. In either case, it is better to disable compression in Kafka since it adds overhead without providing any additional benefit. For messages transmitted over an insecure transport layer, known security exploits of compressed encrypted messages must also be taken into account.

In many environments, especially when TLS is used as the transport layer, message keys do not require encryption since they typically do not contain sensitive data like message payloads. But in some cases, clear-text keys may not comply with regulatory requirements. Since message keys are used for partitioning and compaction, transformation of keys must preserve the required hash equivalence to ensure that a key retains the same hash value even if encryption parameters are altered. One approach would be to store a secure hash of the original key as the message key and store the encrypted message key in the message payload or in a header. Since Kafka serializes message key and value independently, a producer interceptor can be used to perform this transformation.

Authorization

Authorization is the process that determines what operations you are allowed to perform on which resources. Kafka brokers manage access control using a customizable authorizer. We saw earlier that whenever connections are established from a client to a broker, the broker authenticates the client and associates a `KafkaPrincipal` that represents the client identity with the connection. When a request is processed, the broker verifies that the principal associated with the connection is authorized to perform that request. For example, when Alice's producer attempts to write a new customer order record to the topic `customerOrders`, the broker verifies that `User:Alice` is authorized to write to that topic.

Kafka has a built-in authorizer, `AclAuthorizer`, that can be enabled by configuring the authorizer class name as follows:

```
authorizer.class.name=kafka.security.authorizer.AclAuthorizer
```

SIMPLEACLAUTHORIZER

`AclAuthorizer` was introduced in Apache Kafka 2.3. Older versions from 0.9.0.0 onward had a built-in authorizer, `kafka.security.auth.SimpleAclAuthorizer`, which has been deprecated but is still supported.

AclAuthorizer

`AclAuthorizer` supports fine-grained access control for Kafka resources using access control lists (ACLs). ACLs are stored in ZooKeeper and cached in memory by every broker to enable high-performance lookup for authorizing requests. ACLs are loaded into the cache when the broker starts up, and the cache is kept up-to-date using notifications based on a ZooKeeper watcher. Every Kafka request is authorized by verifying that the `KafkaPrincipal` associated with the connection has permissions to perform the requested operation on the requested resources.

Each ACL binding consists of:

- Resource type:
`Cluster|Topic|Group|TransactionalId|DelegationToken`
- Pattern type: `Literal|Prefixed`
- Resource name: Name of the resource or prefix, or the wildcard `*`
- Operation:
`Describe|Create|Delete|Alter|Read|Write|DescribeConfigs|AlterConfigs`
- Permission type: `Allow|Deny`; `Deny` has higher precedence.
- Principal: Kafka principal represented as `<principalType>:<principalName>`, e.g., `User:Bob` or `Group:Sales`. ACLs may use `User:*` to grant access to all users.
- Host: Source IP address of the client connection or `*` if all hosts are authorized.

For example, an ACL may specify:

```
User:Alice has Allow permission for Write to Prefixed Topic:customer from 192.168.0.1
```

`AclAuthorizer` authorizes an action if there are no `Deny` ACLs that match the action and there is at least one `Allow` ACL that matches the action. `Describe` permission is implicitly granted if `Read`, `Write`, `Alter`, or `Delete` permission is granted. `DescribeConfigs` permission is implicitly granted if `AlterConfigs` permission is granted.

WILDCARD ACLS

ACLs with pattern type `Literal` and resource name `*` are used as wildcard ACLs that match all resource names of a resource type.

Brokers must be granted `Cluster:ClusterAction` access in order to authorize controller requests and replica fetch requests. Producers require `Topic:Write` for producing to a topic. For idempotent produce without transactions, producers must also be granted `Cluster:IdempotentWrite`. Transactional producers require `TransactionalId:Write` access to the transaction IS and `Group:Read` for consumer groups to commit offsets. Consumers require `Topic:Read` to consume from a topic and `Group:Read` for the consumer group if using group management or offset management. Administrative operations require appropriate `Create`, `Delete`, `Describe`, `Alter`, `DescribeConfigs`, or `AlterConfigs` access. [Table 11-1](#) shows the Kafka requests to which each ACL is applied.

Table 11-1. Access granted for each Kafka ACL

ACL	Kafka requests	Notes
Cluster:ClusterAction	Inter-broker requests, including controller requests and follower fetch requests for replication	Should only be granted to brokers.
Cluster:Create	CreateTopics and auto-topic creation	Use Topic:Create for fine-grained access control to create specific topics.
Cluster:Alter	CreateAcls, DeleteAcls, AlterReplicaLogDirs, ElectReplicaLeader, AlterPartitionReassignments	
Cluster:AlterConfigs	AlterConfigs and IncrementalAlterConfigs for broker and broker logger, AlterClientQuotas	
Cluster:Describe	DescribeAcls, DescribeLogDirs, ListGroups, ListPartitionReassignments, describing authorized operations for cluster in Metadata request	Use Group:Describe for fine-grained access control for ListGroups.
Cluster:DescribeConfigs	DescribeConfigs for broker and broker logger, DescribeClientQuotas	
Cluster:IdempotentWrite	Idempotent InitProducerId and Produce requests	Only required for non-transactional idempotent producers.
Topic:Create	CreateTopics and auto-topic creation	
Topic:Delete	DeleteTopics, DeleteRecords	
Topic:Alter	CreatePartitions	
Topic:AlterConfigs	AlterConfigs and IncrementalAlterConfigs for topics	

ACL	Kafka requests	Notes
Topic:Describe	Metadata request for topic, OffsetForLeaderEpoch, ListOffset, OffsetFetch	
Topic:DescribeConfigs	DescribeConfigs for topics, for returning configs in CreateTopics response	
Topic:Read	Consumer Fetch, OffsetCommit, TxnOffsetCommit, OffsetDelete	Should be granted to consumers.
Topic:Write	Produce, AddPartitionsToTxn	Should be granted to producers.
Group:Read	JoinGroup, SyncGroup, LeaveGroup, Heartbeat, OffsetCommit, AddOffsetsToTxn, TxnOffsetCommit	Required for consumers using consumer group management or Kafka-based offset management. Also required for transactional producers to commit offsets within a transaction.
Group:Describe	FindCoordinator, DescribeGroup, ListGroups, OffsetFetch	
Group:Delete	DeleteGroups, OffsetDelete	
TransactionalId:Write	Produce and InitProducerId with transactions, AddPartitionToTxn, AddOffsetsToTxn, TxnOffsetCommit, EndTxn	Required for transactional producers.
TransactionalId:Describe	FindCoordinator for transaction coordinator	
DelegationToken:Describe	DescribeTokens	

Kafka provides a tool for managing ACLs using the authorizer configured in brokers. ACLs can be created directly in ZooKeeper as well. This is useful to create broker ACLs prior to starting brokers:

```
$ bin/kafka-acls.sh --add --cluster --operation ClusterAction \
  --authorizer-properties zookeeper.connect=localhost:2181 \
  --allow-principal User:kafka
$ bin/kafka-acls.sh --bootstrap-server localhost:9092 \
```

```

--command-config admin.props --add --topic broad customerOrders \ ❷
--producer --allow-principal User:Alice
$ bin/kafka-acls.sh --bootstrap-server localhost:9092 \
--command-config admin.props --add --resource-pattern-type PREFIXED \ ❸
--topic customer --operation Read --allow-principal User:Bob

```

- ❶ ACLs for broker user are created directly in ZooKeeper.
- ❷ By default, the ACLs command grants literal ACLs. `User:Alice` is granted access to write to the topic `customerOrders`.
- ❸ The prefixed ACL grants permission for Bob to read all topics starting with `customer`.

`AclAuthorizer` has two configuration options to grant broad access to resources or principals in order to simplify management of ACLs, especially when adding authorization to existing clusters for the first time:

```

super.users=User:Carol;User:Admin
allow.everyone.if.no.acl.found=true

```

Super users are granted access for all operations on all resources without any restrictions and cannot be denied access using `Deny` ACLs. If Carol's credentials are compromised, Carol must be removed from `super.users`, and brokers must be restarted to apply the changes. It is safer to grant specific access using ACLs to users in production systems to ensure access can be revoked easily, if required.

SUPER USER SEPARATOR

Unlike other list configurations in Kafka that are comma-separated, `super.users` are separated by a semicolon since user principals such as distinguished names from SSL certificates often contain commas.

If `allow.everyone.if.no.acl.found` is enabled, all users are granted access to resources without any ACLs. This option may be useful when enabling authorization for the first time in a cluster or during development, but is not suitable for production use since access may be granted unintentionally to new resources. Access may also be unexpectedly removed when ACLs for a matching prefix or wildcard are added if the condition for `no.acl.found` no longer applies.

Customizing Authorization

Authorization can be customized in Kafka to implement additional restrictions or add new types of access control, like role-based access control.

The following custom authorizer restricts usage of some requests to the internal listener alone. For simplicity, the requests and listener name are hard-coded here, but they can be configured using custom authorizer properties instead for flexibility:

```

public class CustomAuthorizer extends AclAuthorizer {
    private static final Set<Short> internalOps =
        Utils.mkSet(CREATE_ACLS.id, DELETE_ACLS.id);
    private static final String internalListener = "INTERNAL";

    @Override

```

```

public List<AuthorizationResult> authorize(
    AuthorizableRequestContext context, List<Action> actions) {
    if (!context.listenerName().equals(internalListener) && ❶
        internalOps.contains((short) context.requestType()))
        return Collections.nCopies(actions.size(), DENIED);
    else
        return super.authorize(context, actions); ❷
    }
}

```

- ❶ Authorizers are given the request context with metadata that includes listener names, security protocol, request types, etc., enabling custom authorizers to add or remove restrictions based on the context.
- ❷ We reuse functionality from the built-in Kafka authorizer using the public API.

Kafka authorizer can also be integrated with external systems to support group-based access control or role-based access control. Different principal types can be used to create ACLs for group principals or role principals. For instance, roles and groups from an LDAP server can be used to periodically populate `groups` and `roles` in the Scala class below to support Allow ACLs at different levels:

```

class RbacAuthorizer extends AclAuthorizer {

    @volatile private var groups = Map.empty[KafkaPrincipal, Set[KafkaPrincipal]]
        .withDefaultValue(Set.empty) ❶
    @volatile private var roles = Map.empty[KafkaPrincipal, Set[KafkaPrincipal]]
        .withDefaultValue(Set.empty) ❷

    override def authorize(context: AuthorizableRequestContext,
        actions: util.List[Action]): util.List[AuthorizationResult] = {
        val principals = groups(context.principal) + context.principal
        val allPrincipals = principals.flatMap(roles) ++ principals ❸
        val contexts = allPrincipals.map(authorizeContext(context, _))
        actions.asScala.map { action =>
            val authorized = contexts.exists(
                super.authorize(_, List(action).asJava).get(0) == ALLOWED)
            if (authorized) ALLOWED else DENIED ❹
        }.asJava
    }

    private def authorizeContext(context: AuthorizableRequestContext,
        contextPrincipal: KafkaPrincipal): AuthorizableRequestContext = {
        new AuthorizableRequestContext { ❺
            override def principal() = contextPrincipal
            override def clientId() = context.clientId
            override def requestType() = context.requestType
            override def requestVersion() = context.requestVersion
            override def correlationId() = context.correlationId
            override def securityProtocol() = context.securityProtocol
            override def listenerName() = context.listenerName
            override def clientAddress() = context.clientAddress
        }
    }
}

```

- ❶ Groups to which each user belongs, populated from an external source like LDAP.
- ❷ Roles associated with each user, populated from an external source like LDAP.

- ❸ We perform authorization for the user as well as for all the groups and roles of the user.
- ❹ If any of the contexts are authorized, we return `ALLOWED`. Note that this example doesn't support `Deny` ACLs for groups or roles.
- ❺ We create an authorization context for each principal with the same metadata as the original context.

ACLs can be assigned for the group `Sales` or the role `Operator` using the standard Kafka ACL tool:

```
$ bin/kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin.props --add --topic customer --producer \
  --resource-pattern-type PREFIXED --allow-principal Group:Sales ❶
$ bin/kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin.props --add --cluster --operation Alter \
  --allow-principal=Role:Operator ❷
```

- ❶ We use the principal `Group:Sales` with the custom principal type `Group` to create an ACL that applies to users belonging to the group `Sales`.
- ❷ We use the principal `Role:Operator` with the custom principal type `Role` to create an ACL that applies to users with the role `Operator`.

Security Considerations

Since `AclAuthorizer` stores ACLs in ZooKeeper, access to ZooKeeper should be restricted. Deployments without a secure ZooKeeper can implement custom authorizers to store ACLs in a secure external database.

In large organizations with a large number of users, managing ACLs for individual resources may become very cumbersome. Reserving different resource prefixes for different departments enables the use of prefixed ACLs that minimize the number of ACLs required. This can be combined with group- or role-based ACLs, as shown in the example earlier, to further simplify access control in large deployments.

Restricting user access using the principle of least privilege can limit exposure if a user is compromised. This means granting access only to the resources necessary for each user principal to perform their operations, and removing ACLs when they are no longer required. ACLs should be removed immediately when a user principal is no longer in use, for instance, when a person leaves the organization. Long-running applications can be configured with service credentials rather than credentials associated with a specific user to avoid any disruption when employees leave the organization. Since long-lived connections with a user principal may continue to process requests even after the user has been removed from the system, `Deny` ACLs can be used to ensure that the principal is not unintentionally granted access through ACLs with wildcard principals. Reuse of principals must be avoided if possible to prevent access from being granted to connections using the older version of a principal.

Auditing

Kafka brokers can be configured to generate comprehensive *log4j* logs for auditing and debugging. The logging level as well as the appenders used

for logging and their configuration options can be specified in *log4j.properties*. The logger instances `kafka.authorizer.logger` used for authorization logging and `kafka.request.logger` used for request logging can be configured independently to customize the log level and retention for audit logging. Production systems can use frameworks like the Elastic Stack to analyze and visualize these logs.

Authorizers generate `INFO`-level log entries for every attempted operation for which access was denied, and log entries at the `DEBUG` level for every operation for which access was granted. For example:

```
DEBUG Principal = User:Alice is Allowed Operation = Write from host = 127.0.0.1 on resou
INFO Principal = User:Mallory is Denied Operation = Describe from host = 10.0.0.13 on re
```

Request logging generated at the `DEBUG` level also includes details of the user principal and client host. Full details of the request are included if the request logger is configured to log at the `TRACE` level. For example:

```
DEBUG Completed request:RequestHeader(apiKey=PRODUCE, apiVersion=8, clientId=producer-1,
```

Authorizer and request logs can be analyzed to detect suspicious activities. Metrics that track authentication failures, as well as authorization failure logs, can be extremely useful for auditing and provide valuable information in the event of an attack or unauthorized access. For end-to-end auditability and traceability of messages, audit metadata can be included in message headers when messages are produced. End-to-end encryption can be used to protect the integrity of this metadata.

Securing ZooKeeper

ZooKeeper stores Kafka metadata that is critical for maintaining the availability of Kafka clusters, and hence it is vital to secure ZooKeeper in addition to securing Kafka. ZooKeeper supports authentication using SASL/GSSAPI for Kerberos authentication and SASL/DIGEST-MD5 for username/password authentication. ZooKeeper also added TLS support in 3.5.0, enabling mutual authentication as well as encryption of data in transit. Note that SASL/DIGEST-MD5 should only be used with TLS encryption and is not suitable for production use due to known security vulnerabilities.

SASL

SASL configuration for ZooKeeper is provided using the Java system property `java.security.auth.login.config`. The property must be set to a JAAS configuration file that contains a login section with the appropriate login module and its options for the ZooKeeper server. Kafka brokers must be configured with the client-side login section for ZooKeeper clients to talk to SASL-enabled ZooKeeper servers. The `Server` section that follows provides the JAAS configuration for the ZooKeeper server to enable Kerberos authentication:

```
Server {
    com.sun.security.auth.module.Krb5LoginModule required
    useKeyTab=true storeKey=true
    keyTab="/path/to/zk.keytab"
    principal="zookeeper/zk1.example.com@EXAMPLE.COM";
};
```

To enable SASL authentication on ZooKeeper servers, configure authentication providers in the ZooKeeper configuration file:

```
authProvider.sasl=org.apache.zookeeper.server.auth.SASLAuthenticationProvider
kerberos.removeHostFromPrincipal=true
kerberos.removeRealmFromPrincipal=true
```

BROKER PRINCIPAL

By default, ZooKeeper uses the full Kerberos principal, e.g., `kafka/broker1.example.com@EXAMPLE.COM`, as the client identity. When ACLs are enabled for ZooKeeper authorization, ZooKeeper servers should be configured with `kerberos.removeHostFromPrincipal=true` and `kerberos.removeRealmFromPrincipal=true` to ensure that all brokers have the same principal.

Kafka brokers must be configured to authenticate to ZooKeeper using SASL with a JAAS configuration file that provides client credentials for the broker:

```
Client {
    com.sun.security.auth.module.Krb5LoginModule required
    useKeyTab=true storeKey=true
    keyTab="/path/to/broker1.keytab"
    principal="kafka/broker1.example.com@EXAMPLE.COM";
};
```

SSL

SSL may be enabled on any ZooKeeper endpoint, including those that use SASL authentication. Like Kafka, SSL may be configured to enable client authentication, but unlike Kafka, connections with both SASL and SSL client authentication authenticate using both protocols and associate multiple principals with the connection. ZooKeeper authorizer grants access to a resource if any of the principals associated with the connection have access.

To configure SSL on a ZooKeeper server, a key store with the hostname of the server or a wildcarded host should be configured. If client authentication is enabled, a trust store to validate client certificates is also required:

```
secureClientPort=2181
serverCnxnFactory=org.apache.zookeeper.server.NettyServerCnxnFactory
authProvider.x509=org.apache.zookeeper.server.auth.X509AuthenticationProvider
ssl.keyStore.location=/path/to/zk.ks.p12
ssl.keyStore.password=zk-ks-password
ssl.keyStore.type=PKCS12
ssl.trustStore.location=/path/to/zk.ts.p12
ssl.trustStore.password=zk-ts-password
ssl.trustStore.type=PKCS12
```

To configure SSL for Kafka connections to ZooKeeper, brokers should be configured with a trust store to validate ZooKeeper certificates. If client authentication is enabled, a key store is also required:

```
zookeeper.ssl.client.enable=true
zookeeper.clientCnxnSocket=org.apache.zookeeper.ClientCnxnSocketNetty
zookeeper.ssl.keystore.location=/path/to/zkclient.ks.p12
zookeeper.ssl.keystore.password=zkclient-ks-password
```

```
zookeeper.ssl.keystore.type=PKCS12
zookeeper.ssl.truststore.location=/path/to/zkclient.ts.p12
zookeeper.ssl.truststore.password=zkclient-ts-password
zookeeper.ssl.truststore.type=PKCS12
```

Authorization

Authorization can be enabled for ZooKeeper nodes by setting ACLs for the path. When brokers are configured with `zookeeper.set.acl=true`, the broker sets ACLs for ZooKeeper nodes when creating the node. By default, metadata nodes are readable by everyone but modifiable only by brokers. Additional ACLs may be added if required for internal admin users who may need to update metadata directly in ZooKeeper. Sensitive paths, like nodes containing SCRAM credentials, are not world-readable by default.

Securing the Platform

In the previous sections, we discussed the options for locking down access to Kafka and ZooKeeper in order to safeguard Kafka deployments. Security design for a production system should use a threat model that addresses security threats not just for individual components but also for the system as a whole. Threat models build an abstraction of the system and identify potential threats and the associated risks. Once the threats are evaluated, documented, and prioritized based on risks, mitigation strategies must be implemented for each potential threat to ensure that the whole system is protected. When assessing potential threats, it is important to consider external threats as well as insider threats. For systems that store Personally Identifiable Information (PII) or other sensitive data, additional measures to comply with regulatory policies must also be implemented. An in-depth discussion of standard threat modeling techniques is outside the scope of this chapter.

In addition to protecting data in Kafka and metadata in ZooKeeper using secure authentication, authorization, and encryption, extra steps must be taken to ensure that the platform is secure. Defenses may include network firewall solutions to protect the network and encryption to protect physical storage. Key stores, trust stores, and Kerberos keytab files that contain credentials used for authentication must be protected using filesystem permissions. Access to configuration files containing security-critical information like credentials must be restricted. Since passwords stored in clear-text in configuration files are insecure even if access is restricted, Kafka supports externalizing passwords in a secure store.

Password Protection

Customizable configuration providers can be configured for Kafka brokers and clients to retrieve passwords from a secure third-party password store. Passwords may also be stored in encrypted form in configuration files with custom configuration providers that perform decryption.

The custom configuration provider that follows uses the tool `gpg` to decrypt broker or client properties stored in a file:

```
public class GpgProvider implements ConfigProvider {

    @Override
    public void configure(Map<String, ?> configs) {}
}
```

```

@Override
public ConfigData get(String path) {
    try {
        String passphrase = System.getenv("PASSPHRASE"); ❶
        String data = Shell.execCommand(
            "gpg", "--decrypt", "--passphrase", passphrase, path); ❷
        Properties props = new Properties();
        props.load(new StringReader(data)); ❸
        Map<String, String> map = new HashMap<>();
        for (String name : props.stringPropertyNames())
            map.put(name, props.getProperty(name));
        return new ConfigData(map);
    } catch (IOException e) {
        throw new RuntimeException(e); ❹
    }
}

@Override
public ConfigData get(String path, Set<String> keys) { ❺
    ConfigData configData = get(path);
    Map<String, String> data = configData.data().entrySet()
        .stream().filter(e -> keys.contains(e.getKey()))
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
    return new ConfigData(data, configData.ttl());
}

@Override
public void close() {}
}

```

- ❶ We provide the passphrase for decoding passwords to the process in the environment variable `PASSPHRASE`.
- ❷ We decrypt the configs using `gpg`. The return value contains the full set of decrypted configs.
- ❸ We parse the configs in `data` as Java properties.
- ❹ We fail fast with a `RuntimeException` if an error is encountered.
- ❺ Caller may request a subset of keys from the path; here we get all values and return the requested subset.

You may recall that in the section on SASL/PLAIN, we used standard Kafka configuration classes to load credentials from an external file. We can now encrypt that file using `gpg`:

```

gpg --symmetric --output credentials.props.gpg \
    --passphrase "$PASSPHRASE" credentials.props

```

We now add indirect configs and config provider options to the original properties file so that Kafka clients load their credentials from the encrypted file:

```

username=${gpg:/path/to/credentials.props.gpg:username}
password=${gpg:/path/to/credentials.props.gpg:password}
config.providers=gpg
config.providers.gpg.class=com.example.GpgProvider

```

Sensitive broker configuration options can also be stored encrypted in ZooKeeper using the Kafka configs tool without using custom providers. The following command can be executed before starting brokers to store

encrypted SSL key store passwords for brokers in ZooKeeper. The password encoder secret must be configured in each broker's configuration file to decrypt the value:

```
$ bin/kafka-configs.sh --zookeeper localhost:2181 --alter \
--entity-type brokers --entity-name 0 --add-config \
'listener.name.external.ssl.keystore.password=server-ks-password,password.encoder.secret'
```

Summary

The frequency and scale of data breaches have been increasing over the last decade as cyberattacks have become increasingly sophisticated. In addition to the significant cost of isolating and resolving breaches and the cost of outages until security fixes have been applied, data breaches may also result in regulatory penalties and long-term damage to brand reputation. In this chapter, we explored the vast array of options available to guarantee the confidentiality, integrity, and availability of data stored in Kafka.

Going back to the example data flow at the start of this chapter, we reviewed the options available for different aspects of security throughout the flow:

Client authenticity

When Alice's client establishes connection to a Kafka broker, a listener using SASL or SSL with client authentication can verify that the connection is really from Alice and not an imposter.

Reauthentication can be configured to limit exposure in case a user is compromised.

Server authenticity

Alice's client can verify that its connection is to the genuine broker using SSL with hostname validation or using SASL mechanisms with mutual authentication, like Kerberos or SCRAM.

Data privacy

Use of SSL to encrypt data in transit protects data from eavesdroppers. Disk or volume encryption protects data at rest even if the disk is stolen. For highly sensitive data, end-to-end encryption provides fine-grained data access control and ensures that cloud providers and platform administrators with physical access to network and disks cannot access the data.

Data integrity

SSL can be used to detect tampering of data over an insecure network. Digital signatures can be included in messages to verify integrity when using end-to-end encryption.

Access control

Every operation performed by Alice, Bob, and even brokers is authorized using a customizable authorizer. Kafka has a built-in authorizer that enables fine-grained access control using ACLs.

Auditability

Authorizer logs and request logs can be used to track operations and attempted operations for auditing and anomaly detection.

A combination of quotas and configuration options to manage connections can be used to protect brokers from denial-of-service attacks. ZooKeeper can be secured using SSL, SASL, and ACLs to ensure that the metadata needed to ensure the availability of Kafka brokers is secure.

With the wide choice of options available for security, choosing the appropriate options for each use case can be a daunting task. We reviewed the security concerns to consider for each security mechanism, and the controls and policies that can be adopted to limit the potential attack surface. We also reviewed the additional measures required to lock down ZooKeeper and the rest of the platform. The standard security technologies supported by Kafka and the various extension points to integrate with the existing security infrastructure in your organization enable you to build consistent security solutions to protect the whole platform.